Studio Nx

Backend(FastAPI/Python) 03 — http_error_handler()

2026-10-12

Backend(FastAPI/Python) 03 — `http_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통: 오류 응답 형식
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	02 validation_error_handler() · 04 unexpected_error_handler() (예상하지 못한 오류 500)

1. 하는 일

이 프로젝트의 코드가 아니라 **FastAPI와 Starlette가 직접 내는 오류**를 다른 오류와 같은 모양으로 바꿉니다.

`GET /api/v1/no-such-endpoint` (없는 API 주소)의 경우:

	응답
바꾸기 전 (FastAPI 기본)	`404 {"detail": "Not Found"}`
바꾼 뒤	`404 {"error": {"code": "NOT_FOUND", "message": "Not found.", "details": null}}`

2. 언제 쓰나

라우터 코드가 실행되지 않았거나, 라이브러리가 스스로 요청을 거절할 때 생깁니다.

상황	상태	code
없는 API 주소 (`/api/v1/projectz`)	404	`NOT_FOUND`
주소는 있는데 방식이 다름 (POST만 받는 `/auth/login`에 GET)	405	`METHOD_NOT_ALLOWED`
라이브러리가 요청 자체를 거절 (깨진 요청 등)	400	`BAD_REQUEST`

이 프로젝트의 코드가 일부러 내는 오류는 `AppError`로 던지고 01이 처리합니다. 이 함수는 그 밖의 기본 오류를 맡습니다.

오류 처리기 네 개의 역할

#	처리기	맡는 오류	오류를 내는 쪽
01	`app_error_handler`	`AppError`	이 프로젝트의 코드 (일부러 냄)
02	`validation_error_handler`	입력 형식 오류	FastAPI (스키마와 비교)
03	**`http_error_handler`**	404, 405 등	FastAPI와 Starlette (주소와 방식)
04	`unexpected_error_handler`	그 밖의 모든 오류	버그, DB 연결 끊김 등 (예상하지 못한 오류)

페이지 주소는 다르게 처리합니다

없는 **페이지** 주소(`/no-such-page`)는 이 함수까지 오지 않습니다. `core/site_files.py`가 먼저 404 오류 페이지(HTML)를 돌려줍니다. API 주소(`/api/...`)만 이 함수로 옵니다.

주소	응답
`/no-such-page`	404 오류 페이지 HTML ("Lost the signal")
`/api/v1/no-such-endpoint`	404 JSON (이 함수)

메시지가 보이는 곳

JSON 응답은 사람이 아니라 **페이지의 JS(`api.js`)가 받습니다.** 화면에 문구를 보여 줄지는 그 페이지의 JS가 정합니다.

단계	일어나는 일
① 서버	`{"error": {"code": "NOT_FOUND", "message": "Not found."}}`로 응답합니다
② `api.js`	`ApiError`로 바꿉니다. `err.code`는 `"NOT_FOUND"`, `err.message`는 `"Not found."`입니다 (JS 02)
③ 페이지 JS	`catch (err)`로 받아서 화면에 표시합니다 (칸 아래 문구, 알림, 모달 안 문구). `code`를 보고 문구를 직접 정할 수도 있습니다

페이지 JS(P1-07 이후)가 아직 없어서 지금은 화면에서 볼 수 없습니다. 다음 방법으로 확인합니다.

확인 방법	위치
브라우저 개발자 도구	`F12` → Network 탭 → 요청 → Response
API 문서 화면	`http://localhost:8000/docs` → API 선택 → Try it out
주소창에 API 주소 입력	`localhost:8000/api/v1/abc`를 입력하면 JSON 문자열이 그대로 보입니다

오류 HTML 페이지(`frontend/errors/*.html`)와의 관계

요청은 두 종류이고, 오류도 요청의 종류에 따라 다르게 돌려줍니다.

	페이지 요청	API 요청
보내는 쪽	**사람**: 주소창 입력, 링크 클릭	**JS**: `api.get(...)`
주소	`/myPage.html`, `/abc`	`/api/v1/...`
오류 응답	**오류 HTML 페이지** (404, 500 등을 디자인한 화면)	**JSON** (01~04 처리기)
사용자가 보는 것	오류 페이지 전체 화면	지금 보던 화면 안의 문구

API 오류에 HTML을 주면 안 됩니다. 가입 버튼을 눌렀는데 이메일이 중복이라는 이유로 화면 전체가 오류 페이지로 바뀌면 안 되기 때문입니다. API는 JSON으로 답하고, 화면은 그대로 둔 채 문구만 보여 줍니다.

경우	지금	할 작업
없는 페이지 `/abc`	✅ `errors/404.html` (`site_files.py`)	없음
없는 API `/api/v1/abc`	✅ 404 JSON (이 처리기)	없음
페이지 요청 중 서버 오류 (500, 403 등)	❌ JSON이 나갑니다	**P1-22**: 처리기에서 주소가 `/api`로 시작하지 않으면 `errors/500.html` 등을 보냅니다

`errors.py`의 `ERROR_PAGES` 표(`500 → errors/500.html` …)가 P1-22에서 쓸 연결 고리입니다. 01~04 처리기에 "API 요청이면 JSON, 페이지 요청이면 HTML"이라는 갈림길만 더하면 됩니다.

API 오류가 나면 JS가 오류 페이지로 보내는 경우 (2026-10-06 결정)

페이지를 열 때 부르는 **본 데이터**를 받지 못하면, 그 페이지는 보여 줄 것이 없습니다. 이때는 JS가 오류 페이지로 이동합니다. 버튼이나 폼 동작의 오류는 화면 안 문구로 보여 줍니다.

오류가 난 요청	예	처리
본 데이터: 404	`studioNx.html?projectId=999` → `GET /projects/999/full`	`location.replace('errors/404.html')`
본 데이터: 403	다른 사람의 프로젝트를 주소로 열기	`location.replace('errors/403.html')`
본 데이터: 500 이상	대시보드나 프로젝트를 불러오다가 서버 오류	`location.replace('errors/500.html')`
본 데이터: 401	로그인이 풀림	오류 페이지가 아니라 `requireLogin()` 흐름 (로그인 모달)
버튼과 폼 동작 (저장, 가입, 이름 변경 ...)	이메일 중복, 이름 변경 실패	화면 안 문구로 보여 주고, 페이지는 그대로 둡니다

`replace`로 이동하는 이유는 JS 07과 같습니다. 뒤로 가기로 오류가 난 페이지에 다시 들어가지 않게 하기 위해서입니다.

적용할 곳 (각 함수의 TODO 주석에 적어 두었습니다)	본 데이터
시퀀서 `sequencer.js`의 `loadProject()`	`GET /projects/{id}/full`
마이페이지 `mypage.js`의 `loadDashboard()` (P1-17)	`GET /me/dashboard`
회원 정보 `userinfo.js`의 `loadMe()` (P1-10)	`GET /users/me`
내 프로젝트 `myproject.js`의 `loadProjects()` (P1-14)	`GET /projects`: 목록이라서 404와 403은 없고 500 이상만 처리합니다

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import입니다. 이 함수는 **`Request`, `JSONResponse`, `StarletteHTTPException`**을 사용합니다.

함수

```
HTTP_ERRORS: dict[int, tuple[ErrorCode, str]] = {
    400: (ErrorCode.BAD_REQUEST, "Bad request."),
    401: (ErrorCode.UNAUTHORIZED, "Please sign in."),
    403: (ErrorCode.FORBIDDEN, "You do not have permission."),
    404: (ErrorCode.NOT_FOUND, "Not found."),
    405: (ErrorCode.METHOD_NOT_ALLOWED, "Method not allowed."),
    413: (ErrorCode.FILE_TOO_LARGE, "The file is too large."),
```

```
    415: (ErrorCode.UNSUPPORTED_FILE_TYPE, "Unsupported file type."),
    429: (ErrorCode.TOO_MANY_REQUESTS, "Too many requests. Please try again later."),
}


async def http_error_handler(request: Request, exc: StarletteHTTPException) -> JSONResponse:
    if exc.status_code in HTTP_ERRORS:
        code, message = HTTP_ERRORS[exc.status_code]
    elif exc.status_code >= 500:
        code, message = ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again."
    else:
        code, message = ErrorCode.BAD_REQUEST, "Bad request."
    response = error_json(exc.status_code, code, message)
    if exc.headers:
        response.headers.update(exc.headers)
    return response
```

`register_exception_handlers()`에 등록하는 줄 하나를 더하고, `ErrorCode`에 `BAD_REQUEST`와 `METHOD_NOT_ALLOWED` 두 개를 추가했습니다.

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다 (예: `Request`)
`import A as B`	가져오면서 이름을 `B`로 바꿉니다
`HTTPException as StarletteHTTPException`	FastAPI에도 `HTTPException`이 있어서, Starlette 쪽이라는 것을 이름으로 드러냅니다

식	뜻
`dict[int, tuple[ErrorCode, str]]`	키는 정수(상태 코드)이고, 값은 `(code, 문구)` 두 개로 된 묶음입니다
`exc.status_code in HTTP_ERRORS`	사전에 그 키가 있는지 확인합니다
`code, message = HTTP_ERRORS[...]`	두 개 묶음을 두 변수에 나눠 담습니다 (**언패킹**)
`if … elif … else`	순서대로 확인합니다. `elif`는 JS의 `else if`입니다
`response.headers.update(exc.headers)`	오류에 딸린 헤더를 응답에 덧붙입니다
`StarletteHTTPException`	FastAPI가 쓰는 Starlette의 `HTTPException`입니다. FastAPI의 `HTTPException`은 이것을 물려받으므로, 둘 다 이 처리기로 옵니다

상태	`code`	`message`
404	`NOT_FOUND`	`Not found.`
405	`METHOD_NOT_ALLOWED`	`Method not allowed.`
418 (표에 없고 500 미만)	`BAD_REQUEST`	`Bad request.`
502 (표에 없고 500 이상)	`INTERNAL_ERROR`	`Something went wrong. Please try again.`

4. 흐름

1. 상태 코드가 표(`HTTP_ERRORS`)에 있으면 표의 `code`와 문구를 씁니다.
2. 표에 없으면 500 이상은 `INTERNAL_ERROR`, 그 밖은 `BAD_REQUEST`로 정합니다.
3. `error_json()`으로 같은 상태 코드의 응답을 만듭니다 (01).
4. 오류에 헤더가 있으면 응답에 붙입니다.

5. 왜 이렇게

선택	이유
모양을 `{"error": {...}}`로 맞춤	`api.js`는 `error.code`만 읽습니다. 기본 모양인 `{"detail": …}`으로 두면 모든 404가 `INTERNAL_ERROR`로 보입니다 (JS 02 `toApiError`)
문구는 표에 정해 둔 고정 문장	`exc.detail`은 라이브러리나 다른 코드가 만든 문장이라서 내부 정보(서버 주소 등)가 섞일 수 있습니다. 응답에는 정해 둔 문장만 내보냅니다
상태 코드는 그대로 둠	404는 404로 둡니다. 상태를 바꾸면 브라우저, 검색 엔진, 모니터링이 잘못 판단합니다
헤더를 유지함	405 응답에는 받을 수 있는 방식을 알려 주는 `Allow` 헤더가 표준으로 필요합니다
`ErrorCode`에 두 개 추가	405와 그 밖의 400을 나타낼 코드가 없었습니다. 프론트가 분기할 일은 거의 없지만, `code`가 비어 있으면 안 됩니다
401, 403, 413, 415, 429도 표에 둠	지금은 이 프로젝트의 코드가 `AppError`로 내지만, 라이브러리가 같은 상태를 내더라도 같은 `code`가 나오게 합니다

6. 시험 (`tests/test_errors.py`에 4개 추가)

경우	결과
시험 앱의 없는 주소	404이고, 본문이 정확히 `{"error": {"code": "NOT_FOUND", "message": "Not found.", "details": null}}`입니다
GET만 있는 주소에 POST	405 `METHOD_NOT_ALLOWED`이고, `Allow: GET` 헤더가 유지됩니다
418과 502를 내부 문장과 함께 던짐	각각 `BAD_REQUEST`와 `INTERNAL_ERROR`이고, 내부 문장(`10.0.0.5` 등)은 응답에 없습니다
실제 앱의 `/api/v1/no-such-endpoint`와 `/no-such-page`	API는 404 JSON `NOT_FOUND`이고, 페이지는 404 HTML 그대로입니다

전체 시험 24개가 통과했고, `ruff` 검사도 통과했습니다.